

```

import cv2 as cv
import time
import mediapipe as mp
import math
import pyautogui

# using geeks for geeks tutorial to get hands to work
# Grabbing Holistic model from media pipe

pyautogui.PAUSE = 0.01

screen_width, screen_height = pyautogui.size()

mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    max_num_hands = 1,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

#mp_drawing = mp.solutions.drawing_utils

prevTime = 0
currTime = 0

capture = cv.VideoCapture(0)

prev_x, prev_y = pyautogui.position()
"""
for landmark in mp_holistic.HandLandmark:
    print(landmark, landmark.value)

print(mp_holistic.HandLandmark.WRIST.value)
"""

while capture.isOpened():
    # capture frame by frame
    ret, frame = capture.read()

    # This will resize the frame:

```

```

frame = cv.flip(frame, 1)
frame = cv.resize(frame, (640, 480))

# Converts from BGR to RGB
image = cv.cvtColor(frame, cv.COLOR_BGR2RGB)

# This code will have to do with the holistic model
# Makes predictions using holistic model
# This will improve performace, optionally mark the image as not
writeable to pass by refernce
image.flags.writeable = False
results = hands.process(image)

# Converting back to BGR
#image = cv.cvtColor(image, cv.COLOR_RGB2BGR)

# This is for drawing face, which we will ignore for right now.
# We only want left hand at the moment

# Left Hand
"""
mp_drawing.draw_landmarks(
    image,
    results.left_hand_landmarks,
    mp_holistic.HAND_CONNECTIONS
)
"""

# Will measure the distance between index and thumb and print ok
if results.multi_hand_landmarks:
    hand = results.multi_hand_landmarks[0]
    index_finger_tip =
hand.landmark[mp_hands.HandLandmark.INDEX_FINGER_TIP]

    #print((10 * round((wrist.x * 1000) / 10))**1.15, 10 *
round((wrist.y * 500) / 10)**1.15)
    #pyautogui.moveTo(10 * round((wrist.x * 1000) / 10)**1.15, 10 *
round((wrist.y * 500) / 10)**1.15)
    #print(wrist.x * 1000, wrist.y * 1000)
    #pyautogui.moveTo(wrist.x * 1000, wrist.y * 1000)

```

```

target_x = (index_finger_tip.x) * screen_width
target_y = index_finger_tip.y * screen_height

next_x = prev_x + 0.2 * (target_x - prev_x)
next_y = prev_y + 0.2 * (target_y - prev_y)

#print(next_x, next_y)
pyautogui.moveTo(next_x, next_y)
prev_x = next_x
prev_y = next_y

thumb_tip = hand.landmark[mp_hands.HandLandmark.THUMB_TIP]
middle_tip =
hand.landmark[mp_hands.HandLandmark.MIDDLE_FINGER_TIP]

click_distance = math.sqrt( (index_finger_tip.x - thumb_tip.x)**2
+ (index_finger_tip.y - thumb_tip.y)**2 )
if click_distance < 0.02:
    pyautogui.mouseDown(button='left')
    #pyautogui.click()
else:
    pyautogui.mouseUp(button='left')

scroll_distance = math.sqrt( (index_finger_tip.x -
middle_tip.x)**2 + (index_finger_tip.y - middle_tip.y)**2 )

if scroll_distance < 0.02:
    pyautogui.mouseDown(button='middle')
else:
    pyautogui.mouseUp(button='middle')

#cv.imshow("Sup", image)

if cv.waitKey(20) & 0xFF==ord('q'):
    break

capture.release()
cv.destroyAllWindows()

```

```
hands.close()
```

```
# k = cv.waitKey(0) # Waits for keystroke
```